POMOGOTCHI –

PROCESS JOURNAL

DES 222: Responsive Design and Technology

Tereza Prochazkova
t_p111@student.usc.edu.au

Table of Contents

1 Introduction 1

2 Intention and Purpose 1

3 Possible Realisations 1

3.1 Micro:bit 1

3.2 Arduino Nano 1

3.3 ESP32 2

3.4 Pure Web Application 2

4 Features 2

4.1 Hardware (Core) 2

4.1.1 Microcontroller 2

4.1.2 Display 2

4.1.3 Sound 2

4.1.4 Controls 2

4.1.5 Case 3

4.2 Hardware (Extra) 3

4.2.1 Sensors 3

4.2.2 Storage 3

4.2.3 Power 3

4.3 Software 3

4.3.1 Web Interface 3

4.3.2 Integrations 3

4.3.3 Task Manager 4

5 Related Work 4

6 Development 4

6.1 3D Model 4

6.2 Components 7

6.3 Display 7

6.4 Buttons 8

6.5 SD Card 9

6.6 Music 10

6.7 Task Manager 10

6.8 Sensors 11

6.9 Pomodoro Timer 11

6.10 Tamagotchi Pet 11

7 Future Improvements 13

8 Reflection 13

9 Honorary Mention 13

1 Introduction

The Pomogotchi project is a portmanteau (a blending of two words) of the Pomodoro technique and the Tamagotchi concept. The goal of this project is to make focus and productivity more engaging by adding a playful element. The device also supports responsive elements, including environmental monitoring and task tracking.

2 Intention and Purpose

I wanted to make a device that could sit on your desk as a constant reminder to be productive. It is less of a Pomodoro timer and more of a constant motivational reminder, since you can always see it on your desk (unlike an app you need to open, which you only do when you remember).

The Pomodoro element is a perfect tool for generating motivation. As a cat owner, I want to see my pet happy, and this sentiment can be transferred to a digital pet. Making the pet happy by being productive can provide an inconsequential yet sentimental reward for effort, serving as a form of brain stimulation that releases dopamine.

3 Possible Realisations

Different platforms were considered for building this project, with comparison factors including technical capabilities, feasibility, and learning opportunities.

3.1 Micro:bit

The micro:bit is very beginner-friendly with built-in sensors and an LED matrix, but it is lacking in capabilities, such as power and memory, and there are not many learning opportunities, as we have used this microcontroller previously.

3.2 Arduino Nano

The Arduino Nano is more powerful than Micro:bit and (personally) easier to work with, especially with soldering. However, it lacks wireless capabilities, which can be added with extra modules. The ESP32, on the other hand, already includes this feature and is superior in terms of processing speed and memory.

3.3 ESP32

The ESP32-S3 offers significantly more performance than the Micro:bit and Arduino, includes Wi-Fi/Bluetooth capabilities, and has significantly more GPIO pins, which can be used for more demanding devices such as a display. The ESP32 is also capable of hosting a local website for configuration, making it the strongest candidate.

3.4 Pure Web Application

Another option is to make the project a purely web-based application. However, I would like to create something that does not rely on another device, for example, when the task at hand does not require one. Besides, creating a physical device feels more engaging and provides a better learning opportunity.

4 Features

4.1 Hardware (Core)

4.1.1 Microcontroller

The ESP32-S3 Dev Board from Unexpected Maker was chosen for its connectivity and processing power, making it a versatile controller for the whole project.

4.1.2 Display

A 3.5-inch Waveshare LCD will display the Tamagotchi pet, timer, current time, music, tasks, and sensor readings. Its colour capability allows expressive faces and icons.

4.1.3 Sound

An enclosed 8Ω/5W speaker paired with a MAX98357A I2S amplifier provides sound for music and audio cues.

4.1.4 Controls

Tactile buttons give users a simple, reliable way to interact with the device directly, such as starting/stopping sessions, controlling music or navigating tasks.

4.1.5 Case

The case will be 3D-printed, designed to neatly house all components while keeping the device compact and visually appealing on a desk. The case's features and decorations can be swapped out, with various personalisation options.

4.2 Hardware (Extra)

4.2.1 Sensors

A BME280 measures temperature and humidity (it can also measure atmospheric pressure, but that is not used). ENS160 tracks air quality, allowing the device to provide environmental feedback. A light-dependent resistor measures light levels and switches between light and dark backgrounds.

4.2.2 Storage

An SD card module enables logging study data and storing local assets, such as images or audio files.

4.2.3 Power

The device is charged via a USB-C. It was originally intended to feature a 3.7V, 1100mAh LiPo battery and active cooling. However, these were deemed unnecessary and remain as a viable upgrade in the future.

4.3 Software

4.3.1 Web Interface

The ESP32 hosts a lightweight website that serves as a task manager.

4.3.2 Integrations

The web app was also intended to host various integrations (including calendar, notifications, and music playback through Spotify or local playlists). These were not integrated due to time and difficulty constraints, but remain as a viable upgrade in the future.

4.3.3 Task Manager

A built-in task manager enables users to create tasks and check/uncheck them. Tasks can be created on a website and displayed and controlled via the device itself. Future renditions could include grouping tasks into task lists and assigning them to specific pomodoro cycles.

5 Related Work

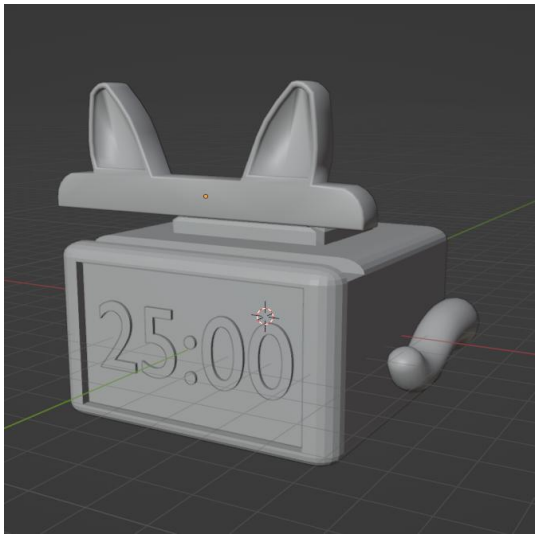
Existing projects include web-based Pomodoro timers, focus apps, and Tamagotchi-related productivity tools (e.g. Forest app – https://www.forestapp.cc). This project was initially inspired by the 'pwnagotchi' tool (https://github.com/evilsocket/pwnagotchi), which learns from its surrounding Wi-Fi environment and captures 3-way handshakes. This project differs by utilising humans' natural emotional attachment to animal pets, transferring it into a motivational drive.

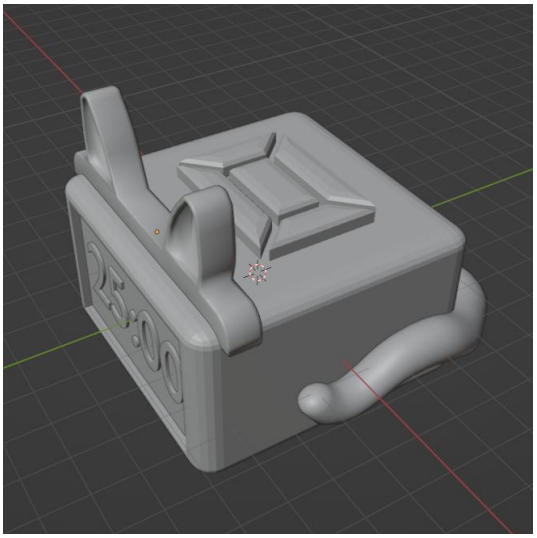
6 Development

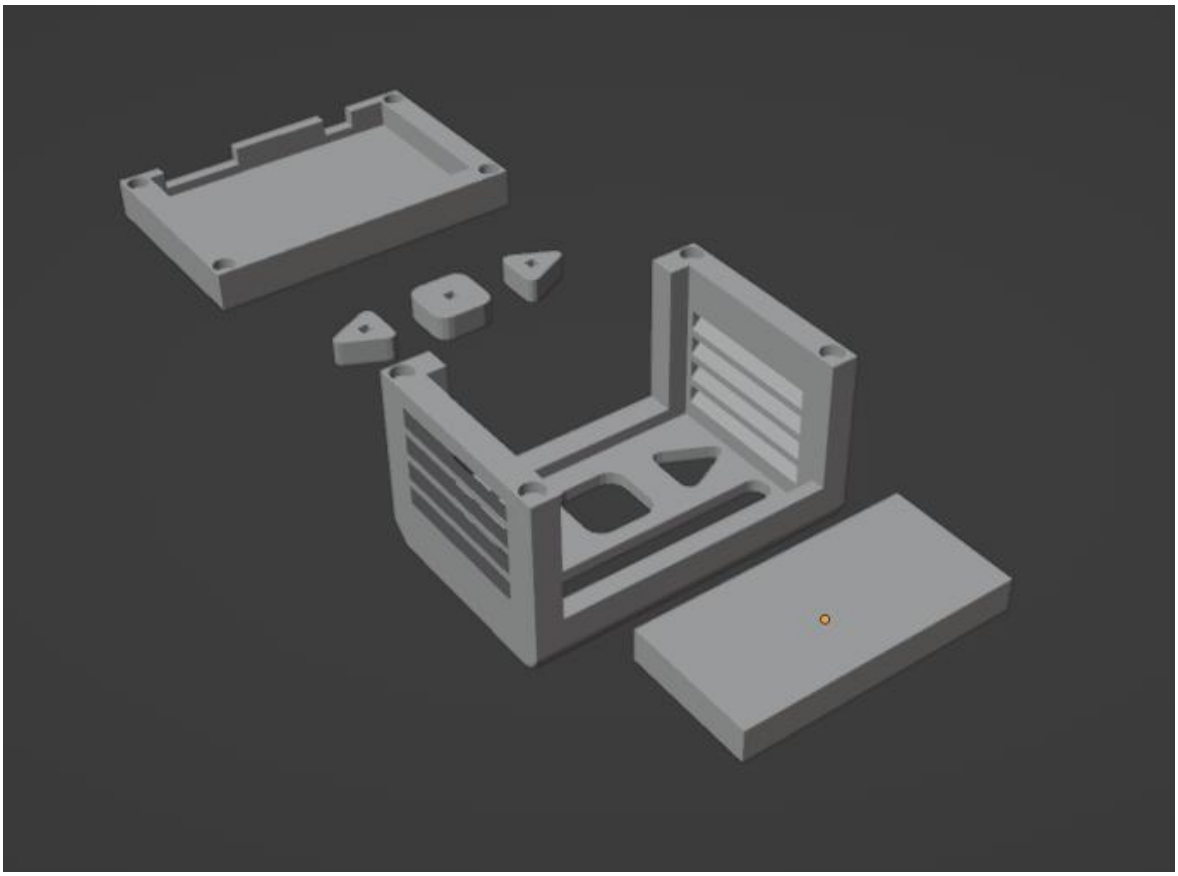
Note that my development process heavily relies on temporary whiteboards, which I photograph, and rapid iterations, which is why I am writing this document in retrospect.

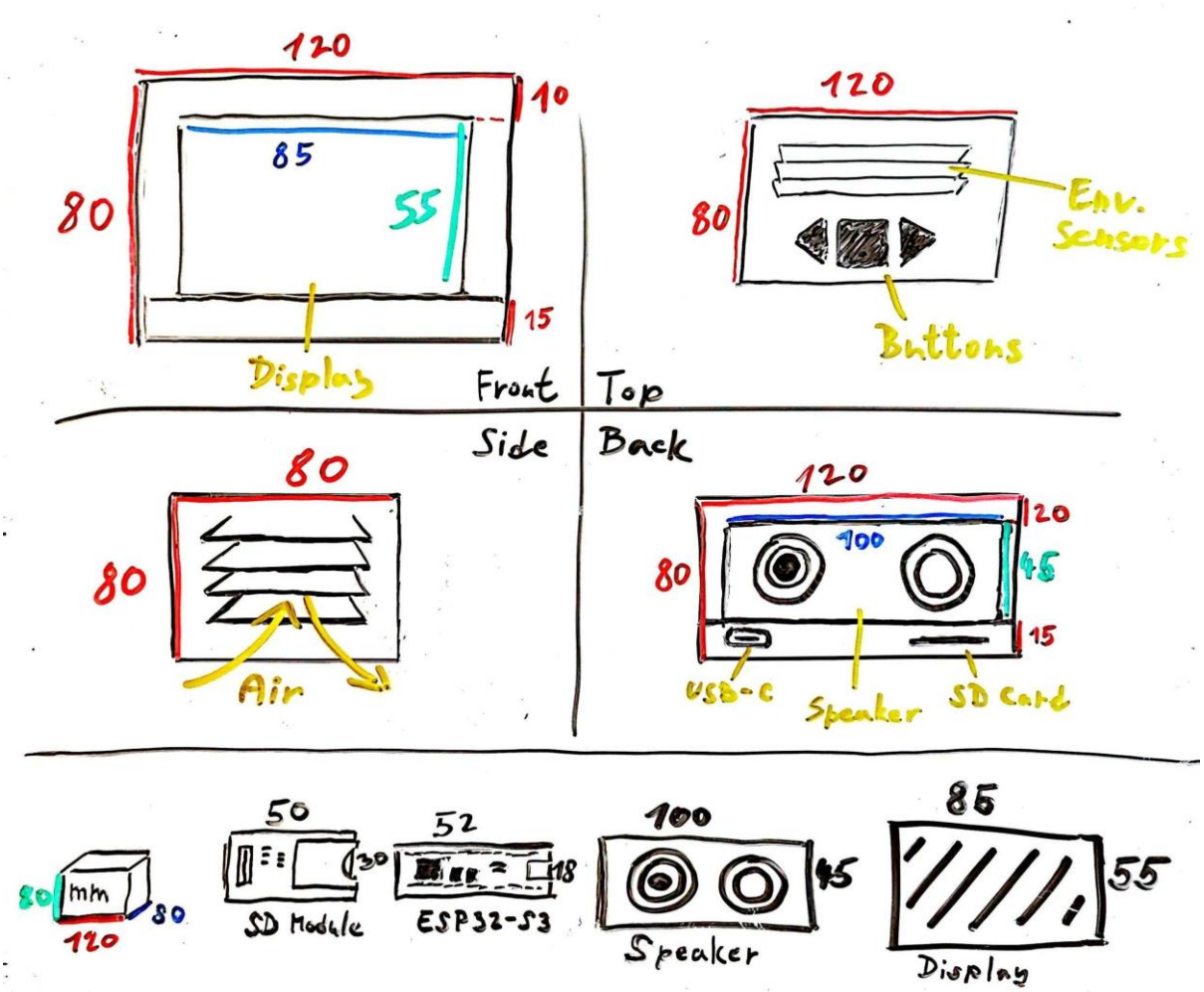
6.1 3D Model

First, I have created the conceptual design of the case, and a rough 3D model of the device's exterior has been developed as a proof-of-concept. Later, I've created the actual 3D printable file that was printed at UniSC using the Bambu Lab X1 Series printer.

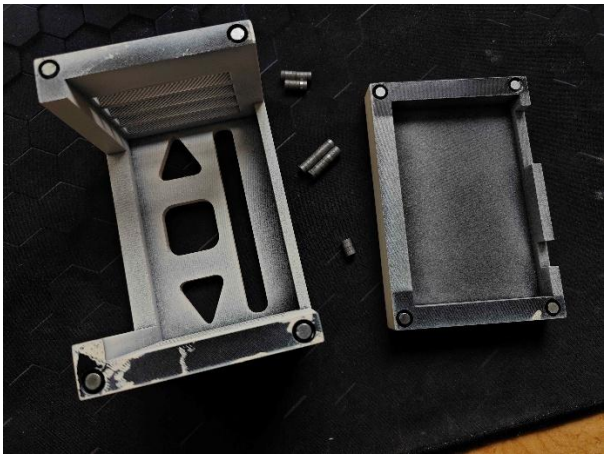

25:00


25:00







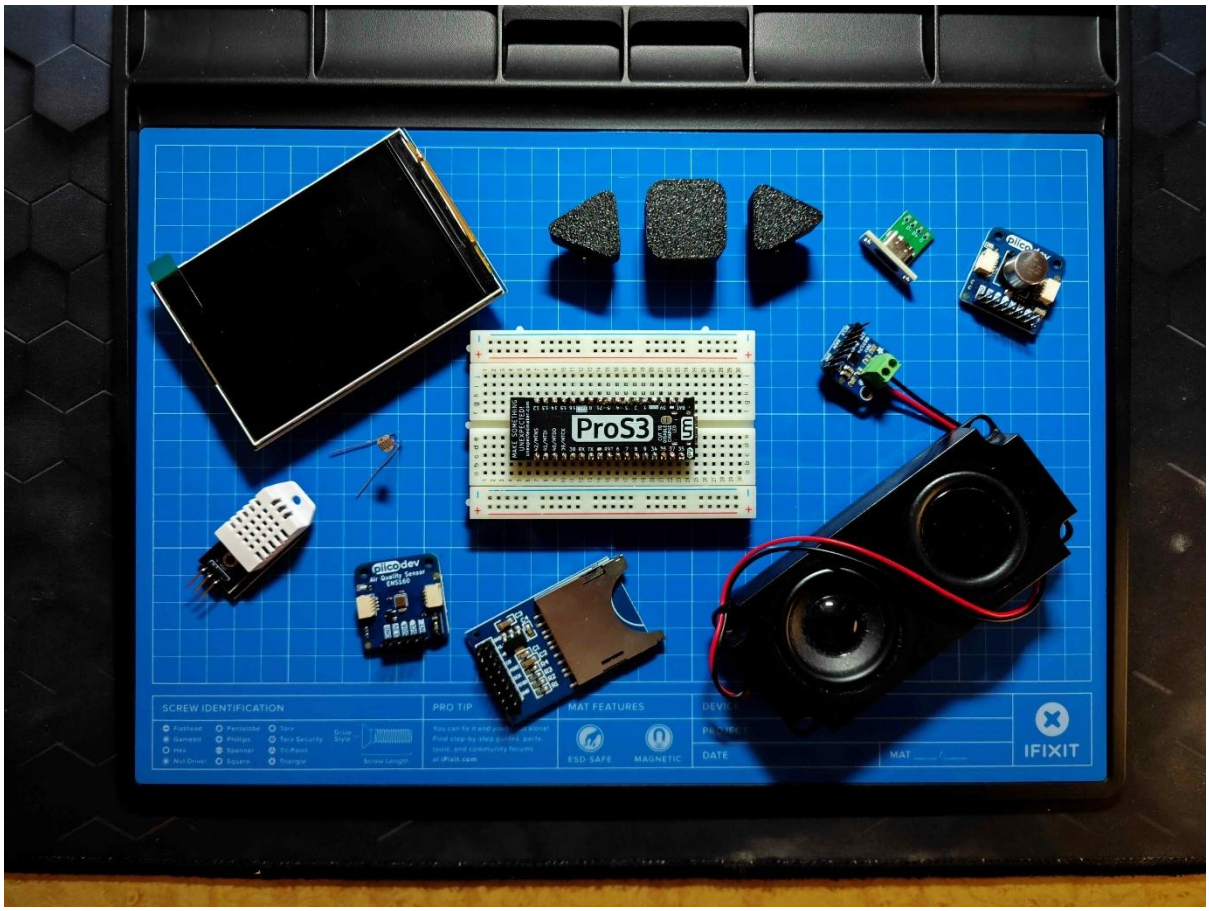


This case includes a front display cutout, a speaker, an SD card slot, a USB-C port at the back, airflow channels with a dust-preventive design on the sides, and a cutout for three buttons at the top. It also includes holes in the corners for magnets, so the case is easily accessible.

As I later realised, this case is too small to accommodate the device with jumper wires. Later, I intend to solder every wire to hopefully decrease the profile and fit the device in the case.

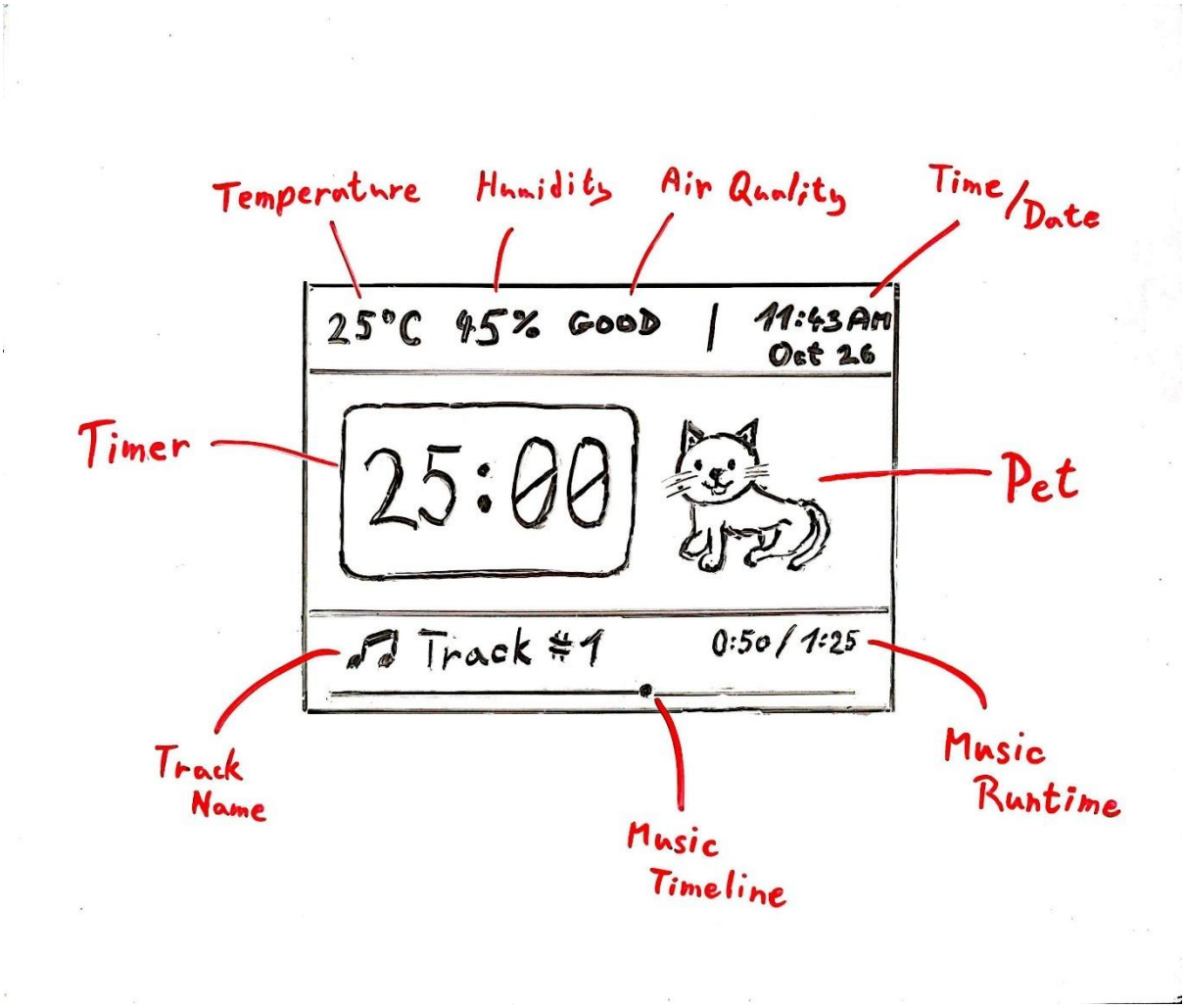
6.2 Components

I have created a list of necessary components and acquired them from various sources (mainly Amazon and Core Electronics). I have verified the compatibility of the chosen components and created a small test script for each to check their functionality and learn how they work.



6.3 Display

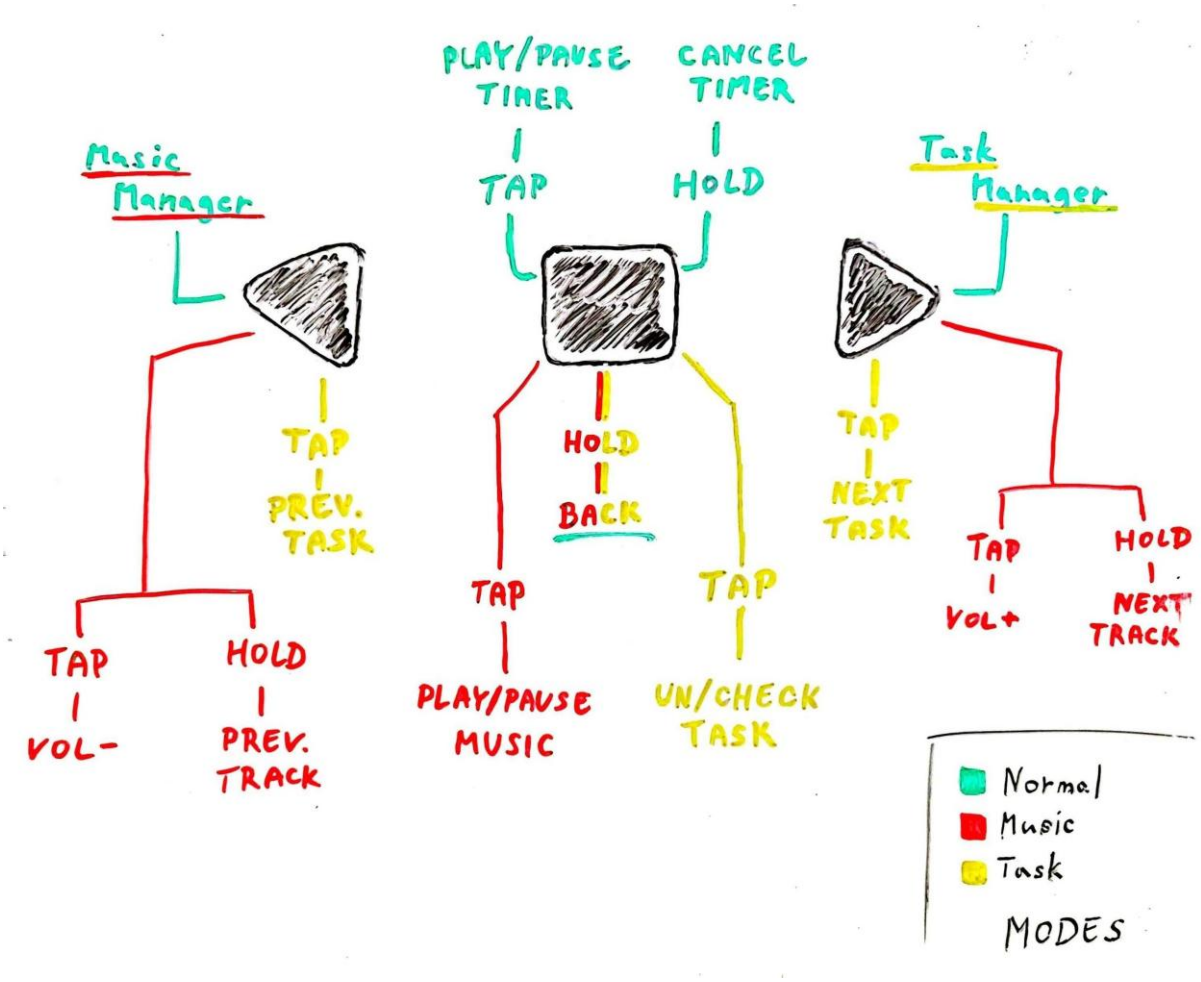
The display has been by far the most challenging component to work with. I started by sketching the UI's rough layout. I then created each element one by one, sourcing data when needed from various sections of the program.



I then created each element one by one, separating the screen into four main segments (top bar, pomodoro timer, Tamagotchi pet, bottom bar), sourcing data when needed from various sections of the program.

6.4 Buttons

The buttons have short- and long-press functionality, and each performs a different function. Below is the diagram of what each button does.



The program functions in three modes: normal, music, and task.

- In Normal Mode, you can start/stop/cancel the pomodoro cycle with the middle button and switch between modes with the left (music) and right (task) buttons.
- In Music Mode, you can play/pause music, as well as change volume and track.
- In Task Mode, you can move between tasks and check/uncheck them.
- Holding the middle button in either Music or Task mode returns to Normal Mode.

6.5 SD Card

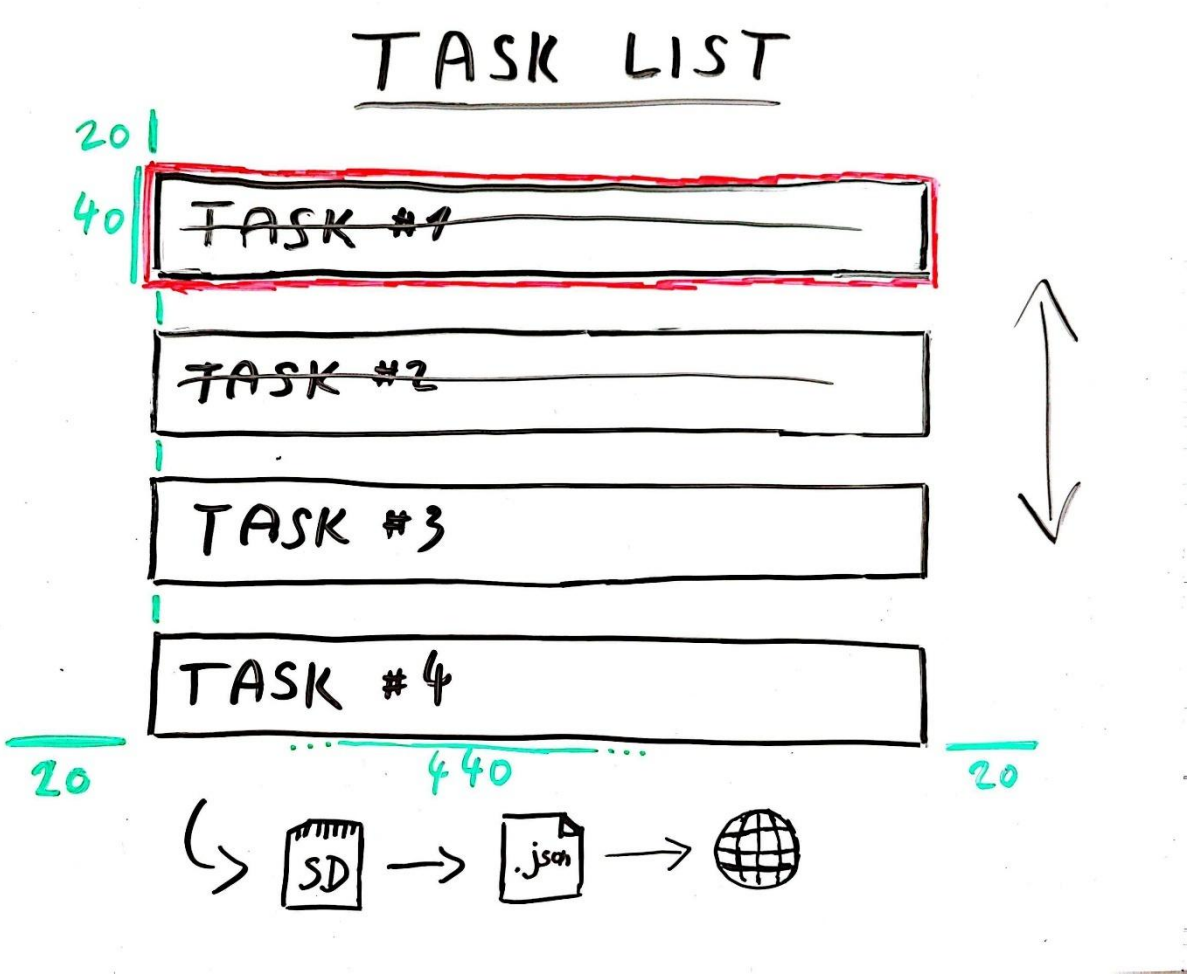
The SD card is a pretty straightforward function. It accesses and reads file on an SD card using the SD.h library. It includes .bmp files for the pet, .mp4 files for the music and notification sounds, and web files with a .json file that stores the tasks.

6.6 Music

Music was also a challenging function as it requires a continuous stream of data with occasional interruptions. As such, this function runs on a separate thread. This, however, created many problems when changing tracks, as it would constantly read a null pointer before another file could be loaded, which took a whole day to debug. Aside from the speaker, it also required an amplifier to boost the volume.

6.7 Task Manager

Currently, tasks are straightforward: they only keep their name, description, and check/uncheck state. These tasks can be added via a website hosted on the ESP32 and checked/unchecked on both the website and the device. In future iterations, I would like to include task lists and the ability to tie tasks to specific pomodoro cycles to track time spent on each task.



6.8 Sensors

This was also a straightforward function and very easy to set up, especially the temperature and humidity sensor, which required only a single line of code.

The Air Quality Sensor, however, was more difficult, as it needed a long time to initialise for the first time. However, once this phase was done, reading the value was as easy as the previous sensor.

The light-dependent resistor was surprisingly more difficult than I thought. Even though the component itself is simple, I had issues with the ESP32's GPIO pins. I ran out of pins by the time I wanted to add the LDR, and the last pin was pin 0. I decided to use that, but every time I did, the device would not turn on at all. After a long debugging session, I learned that pin 0 is an important pin during the ESP32's boot phase and must stay HIGH. I have rectified that by switching the ground and 3.3V for the LDR, so that the pin would read HIGH during boot.

Lastly, the device also includes an RTC module that keeps time and synchronises with an NTP server when Wi-Fi is connected.

6.9 Pomodoro Timer

This was once again an effortless feature. All it does is check millis() every second and, if a second has passed, subtract it from the timer. When the timer runs out, it resets to the duration of the next phase. I originally wanted to use the delay() function, but realised that this would freeze the whole program, unless run on a separate thread.

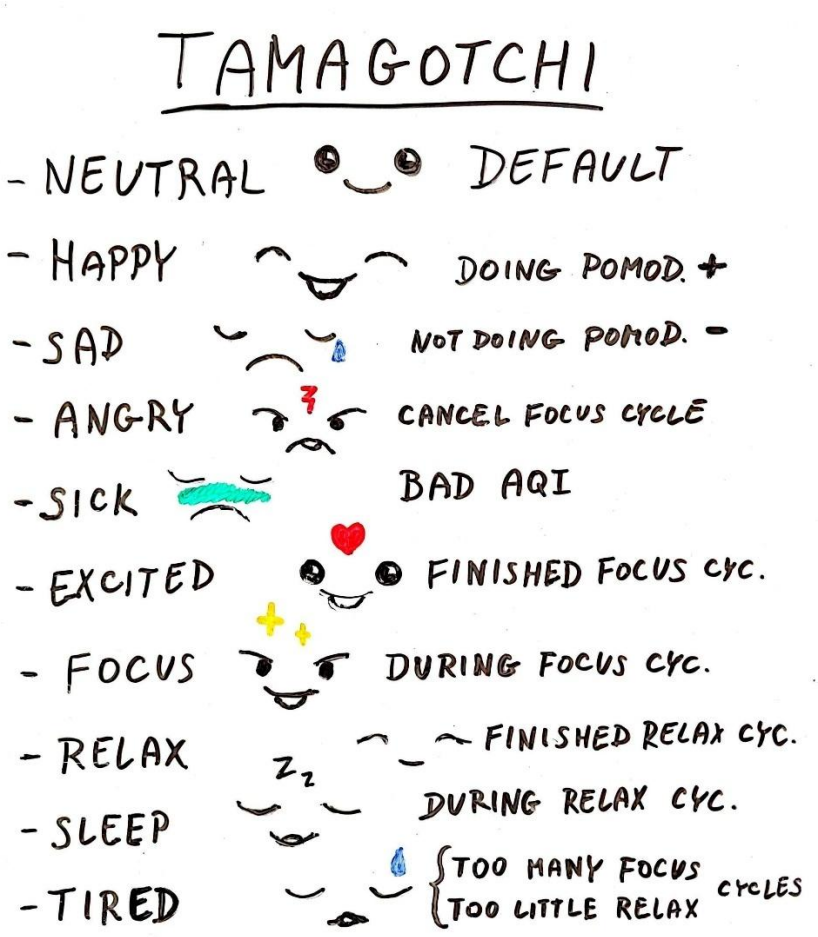
6.10 Tamagotchi Pet

The pet is displayed by overlaying two .bmp images. The first is the pet's base or body (in this case, a cat), and the second is the pet's face or emotion, which constantly changes based on the pet's state. This is designed so that to add a new pet, all that is needed is to create a .bmp file of the new pet with the same dimensions, making them interchangeable.

Below the Tamagotchi pet is a happiness bar that displays the pet's happiness, which increases by finishing pomodoro cycles and decreases with idle time or cancelling pomodoro cycles prematurely.

The emotions are as follows:

- Neutral – The default emotion
- Happy – When the **happiness** is **above** a certain level
- Sad – When the **happiness** is **below** a certain level
- Focused – During the '**focus'** phase of the Pomodoro
- Sleep – During the '**relax'** phase of the Pomodoro
- Excited – After **successfully** finishing the '**focus'** phase
- Relaxed – After **successfully** finishing the '**relax'** phase
- Angry – After **prematurely** cancelling the '**focus'** phase
- Tired – After **prematurely** cancelling the '**relax'** phase



7 Future Improvements

In the future, I would like to add features I did not have time for, such as improved website integration and app integrations (music, calendar, etc.), better customisation options, and enhanced task management.

I also definitely intend to fix the casing and wiring to make it look finished in the near future.

8 Reflection

Overall, I am disappointed with the outcome. Many intended features are missing, and I feel like I have spent too much time for the results I got. Nevertheless, I definitely intend to continue improving the device as I intend to use it regularly. I feel like I have learned a lot from this project, primarily how displays work and how music is streamed from a file.

9 Honorary Mention

My cat.

